

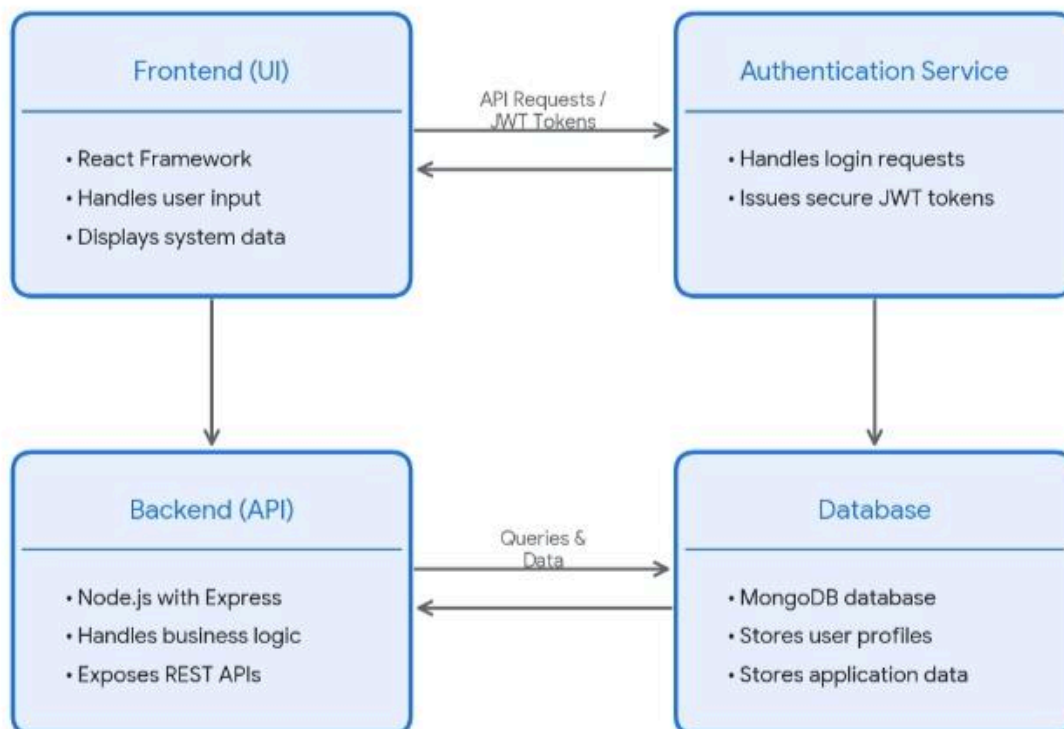
Architecture Image

Project Architecture Description

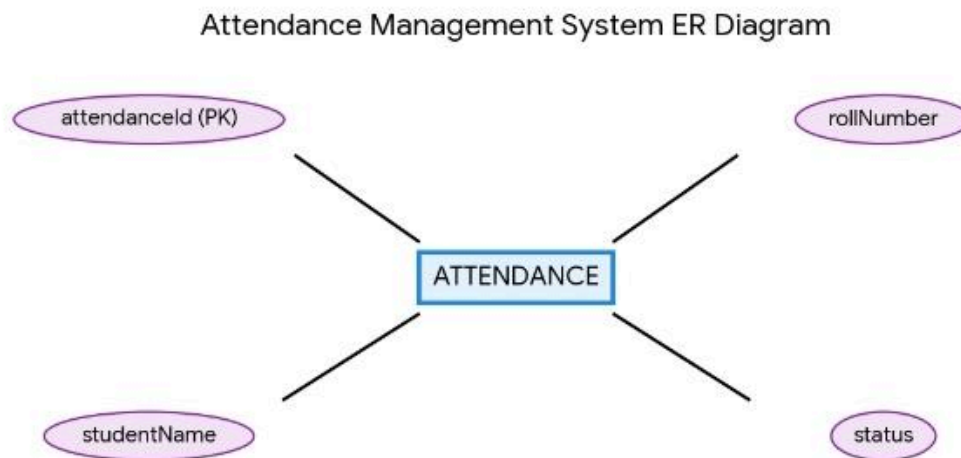
The architecture of the Attendance Management System follows a classic **Full-Stack Decoupled Structure**, separated into four interconnected layers

- **Frontend (UI) Layer:** Developed using the **React Framework**, this layer handles user interactions, captures attendance inputs from students/teachers, and renders data fetched from the backend in an intuitive dashboard.
- **Authentication Service:** A dedicated security layer responsible for processing login credentials and issuing secure **JWT (JSON Web Tokens)** to authorized users for session management.
- **Backend (API) Layer:** Powered by Node.js and Express, this component contains the core business logic, authentication verifications, and exposes secure REST APIs to bridge the client and database.
- **Database Layer:** Uses a scalable **MongoDB database** to store user profiles, roles, security credentials, and daily attendance records securely.

Full-Stack Application Architecture Diagram



ER Diagram



1.database_code.sql

Sql

-- Create Database for Attendance System

```
CREATE DATABASE attendanceDB;
```

```
USE attendanceDB;
```

-- Create Attendance Logs Table

```
CREATE TABLE attendance (  
    attendanceId INT AUTO_INCREMENT PRIMARY KEY,  
    studentName VARCHAR(100) NOT NULL,  
    rollNumber VARCHAR(50) NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    submittedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Insert Sample Data for Testing
INSERT INTO attendance (studentName, rollNumber, status)
VALUES ('Sravani', '23BK1A0501', 'Present');
```

2. Project_Documentation.md

Attendance Management System - Complete Project Documentation

1. Project Overview

This Full-Stack application automates manual attendance processes using modern web architectures. It features an intuitive client dashboard interface connected safely to a localized relational schema storage unit.

2. System Architecture & Diagrams

- **Architecture Diagram**: Built using modular decoupling containing UI layer, Node API server, and MongoDB/SQL configurations.

- **ER Diagram Attributes**: `attendanceId` (Primary Key), `studentName`, `rollNumber`, `status`.

3. Database Execution Schema

The database stores data using a structured table system layout. Logs track whether individual student records match 'Present' or 'Absent' parameters during runtime.

4. Source Code Guide

- Run `npm install` inside the backend repository directory.
- Open `frontend/index.html` to trigger user operations locally.